МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

Практическая работа 2

по дисциплине «Информационные системы управления данными и документами»

Студент группы ИКБО-50-23 Астахов С. П.

_____
(подпись студента)

Принял преподаватель Тесла-Блозва С. А.

_____
(подпись руководителя)

Работа представлена «____»_____ 2026 г.

Допущен к работе «___» _____ 2026 г.

Москва 2026

Оглавление

1. Цель работы 3
2. Ход выполнения работы 3
2.1 Часть 1. Создание проекта на Start.Vaadin 3
2.2 Часть 2 Создание класса модели Document 5
2.3 Часть 3. Создание репозитория и сервиса 8
2.4 Часть 4. Создание экранной формы (View) 9
2.5 Часть 5: Тестирование приложения 12
3. Вывод 17

1. Цель работы

Спроектировать экранную форму создания документа, настроить связывание данных с моделью и реализовать логику проверки вводимых значений

2. Ход выполнения работы

2.1 Часть 1. Создание проекта на Start.Vaadin

Переходим по ссылке: https://start.vaadin.com/app?type=sample и выбираем элементы, представленные в методичке. В GroupID необходимо указать com.astakhov. В ArtifactID указать appikbo_ikbo50(рис. 1):

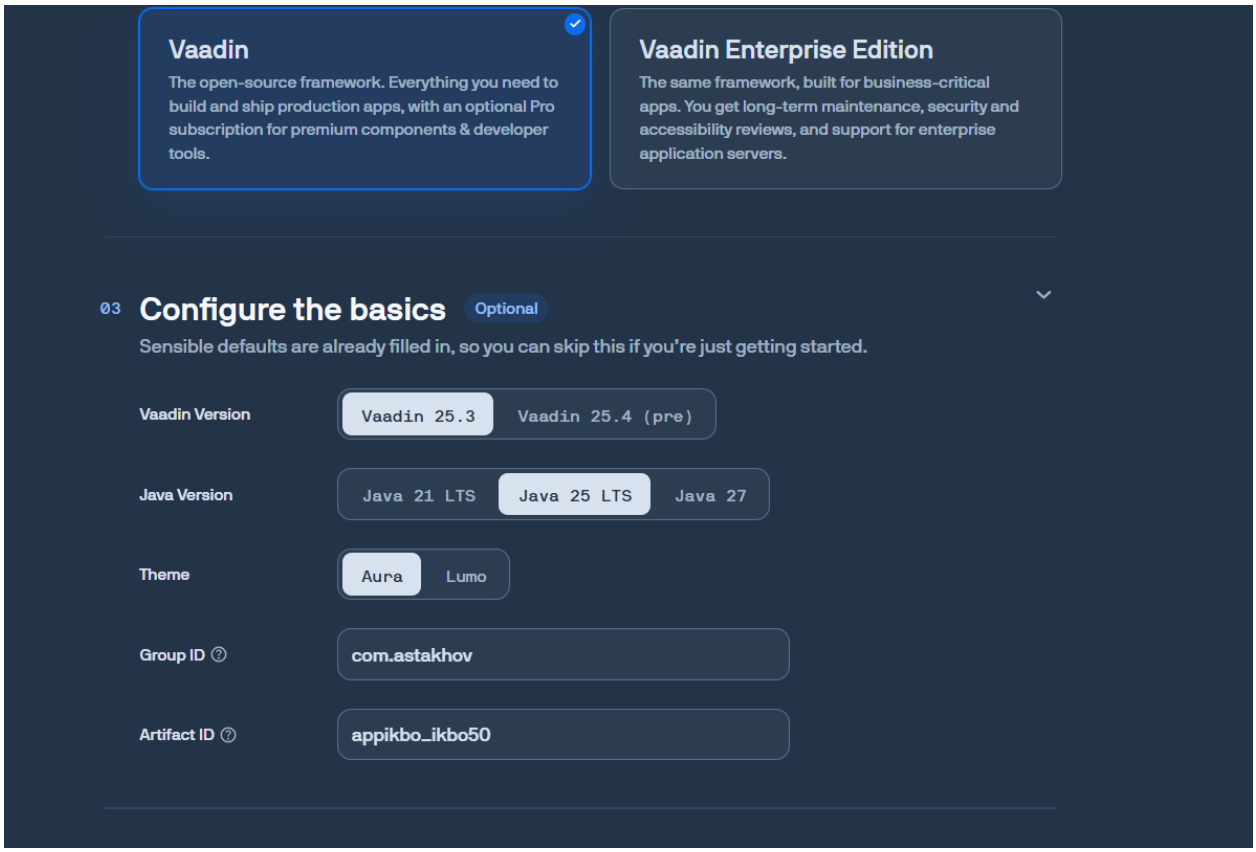


Рисунок 1 – Создание проекта на Start.Vaadin

Скачаем архив с проектом и распакуйте его в директории, где будет создан проект в среде разработки IntelliJ IDEA. Дождаемся окончания индексации проекта и скачивания зависимостей Maven. Перейдем к главному классу приложения по пути: src/main/java/com/astakhov/Application.java и

запускаем проект. IDEA с запущенным проектом и консолью Spring Boot представлен на рисунке 2:

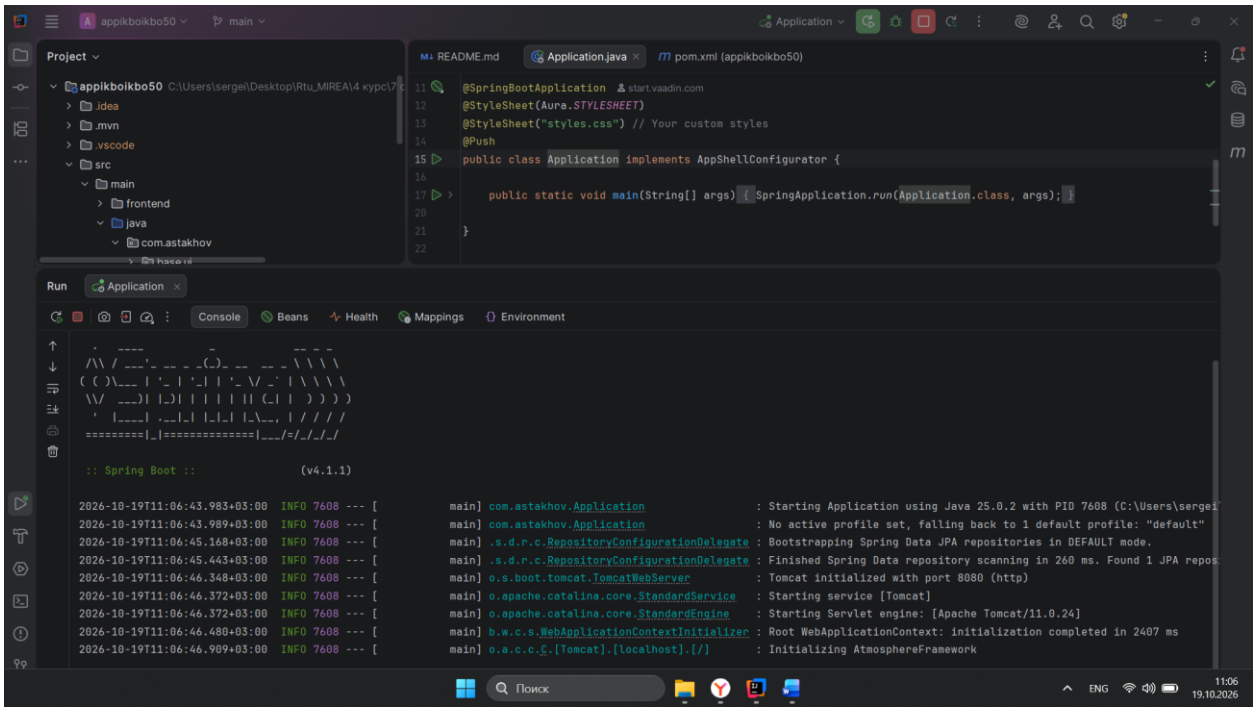


Рисунок 2 – IDEA с запущенным проектом и консолью Spring Boot

После успешного запуска откройте браузер и перейдите по адресу: http://localhost:8080/. Должен открыться базовый интерфейс приложения Vaadin(рисунок 3):

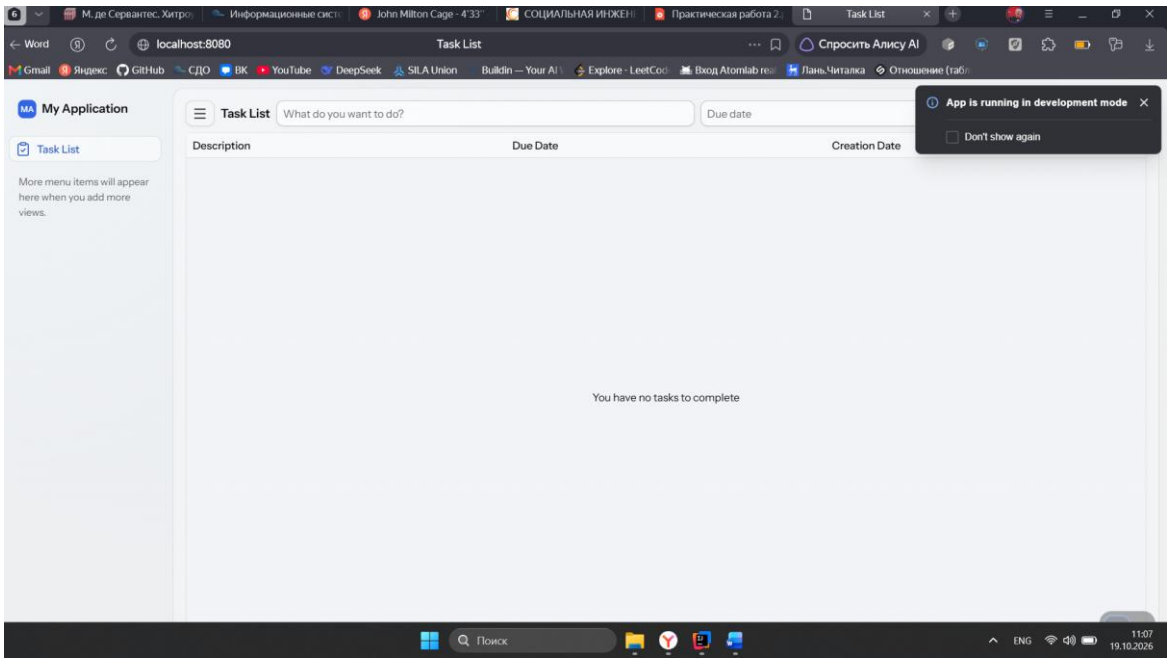


Рисунок 3 – Запуск браузера по локальной ссылке: http://localhost:8080/

2.2 Часть 2 Создание класса модели Document

Создать новый пакет documentfeature внутри вашего основного пакета (com.), а в нем создать класс Document.java. Класс должен содержать следующие поля:

- Название (String);
- Регистрационный номер (String);
- Дата регистрации (LocalDate);
- Тип документа (Enum или String - «Приказ», «Распоряжение», «Служебная записка»).

А также нужно написать Геттеры в IDE командой Alt+Insert -> Getter and Setter.

Пример без getter и setter представлен на листинге 1:

Листинг 1 – Код класса Document.java

```
package com.astakhov.documentfeature;
import java.time.LocalDate;
public class Document {
        private String title;
        private String regNumber;
        private LocalDate regDate;
        private String type; // "Приказ", "Распоряжение", "Служебная записка"
        // Конструкторы
        public Document() {
        }
         public Document(String title, String regNumber, LocalDate regDate, String type) {
                this.title = title;
                this.regNumber = regNumber;
```

```
        this.regDate = regDate;
        this.type = type;
    }
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
}
```

Итоговый код(листинге 2):

Листинг 2 – Итоговый код с getter и setter:

```
package com.astakhov.documentfeature;

import java.time.LocalDate;
public class Document {
   private String title;
   private String regNumber;
   private LocalDate regDate;
   private String type; // "Приказ", "Распоряжение", "Служебная записка"
   // Конструкторы
   public Document() {
   }
   public Document(String title, String regNumber, LocalDate regDate, String type) {
       this.title = title;
       this.regNumber = regNumber;
       this.regDate = regDate;
```

```
    this.type = type;
}
// Getter и Setter
public String getTitle() {
    return title;
}
public void setTitle(String title) {
    this.title = title;
}

public LocalDate getRegDate() {
    return regDate;
}

public void setRegDate(LocalDate regDate) {
    this.regDate = regDate;
}

public String getRegNumber(){
    return regNumber;
}

public void setRegNumber(String regNumber) {
    this.regNumber = regNumber;
}

public String getType() {
    return type;
}
```

```
    public void setType(String type) {
        this.type = type;
    }
}
```

2.3 Часть 3. Создание репозитория и сервиса

Для соблюдения архитектурных принципов Enterprise-разработки создадим классы-заглушки для работы с данными. Создаем интерфейс DocumentRepository.java в пакете documentfeature(Листинг 3) и создадим класс сервиса DocumentService.java в том же пакете(Листинг 4):

Листинг 3 - Интерфейс DocumentRepository.java в пакете documentfeature.

```
package com.astakhov.documentfeature;
import org.springframework.stereotype.Repository;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
@Repository
public class DocumentRepository {
        // Временное хранилище в памяти (для демонстрации)
        private final List<Document> documents = new ArrayList<>();
        private long nextId = 1;
        public Document save(Document document) {
                // Простейшая имитация сохранения
                // В реальности здесь будет работа с БД
                documents.add(document);
                return document;
 }
        public List<Document> findAll() {
                return documents;
```

```
        }
        public Optional<Document> findById(Long id) {
                // Пока не используем ID
                return Optional.empty();
        }
}
```

Листинг 4 - Класс сервиса DocumentService.java в том же пакете

```
package com.astakhov.documentfeature;
import org.springframework.stereotype.Service;
@Service
public class DocumentService {
        private final DocumentRepository documentRepository;
        public DocumentService(DocumentRepository documentRepository) {
                this.documentRepository = documentRepository;
        }
        public Document saveDocument(Document document) {
                // Здесь можно добавить валидацию, логирование и т.д.
                return documentRepository.save(document);
        }
}
```

2.4 Часть 4. Создание экранной формы (View)

Создаем подпакет ui внутри documentfeature, а в нем класс DocumentFormView.java:

```
package com.astakhov.documentfeature.ui;


import com.vaadin.flow.component.button.Button;
```

```
import com.vaadin.flow.component.datepicker.DatePicker;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.notification.Notification;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.binder.Binder;
import com.vaadin.flow.data.validator.RegexpValidator;
import com.vaadin.flow.router.Route;
import com.astakhov.base.ui.MainLayout; // если используете общий лэйаут
import com.astakhov.documentfeature.Document;
import com.astakhov.documentfeature.DocumentService;
import java.time.LocalDate;
@Route(value = "document-form", layout = MainLayout.class) // указываем
URL и лэйаут
public class DocumentFormView extends VerticalLayout {
    private final DocumentService documentService;
    private final Binder<Document> binder = new Binder<>(Document.class);
    // Поля ввода
    private TextField titleField = new TextField("Название");
    private TextField regNumberField = new TextField("Регистрационный
номер");
    private DatePicker regDatePicker = new DatePicker("Дата регистрации");
    private TextField typeField = new TextField("Тип документа"); // заменить
на ComboBox
    private Button saveButton = new Button("Сохранить");
    private Button clearButton = new Button("Очистить");
    public DocumentFormView(DocumentService documentService) {
        this.documentService = documentService;
        // Настройка формы
        FormLayout formLayout = new FormLayout();
```

```
        formLayout.add(titleField, regNumberField, regDatePicker, typeField);
        add(formLayout, saveButton, clearButton);
        // Конфигурация Binder
        binder.forField(titleField)
                .asRequired("Название не может быть пустым")
                .bind(Document::getTitle, Document::setTitle);
        binder.forField(regNumberField)
                .withValidator(new RegexpValidator(
                        "Номер должен соответствовать формату XXX-XX
(например, 123-45)","^\\d{3}-\\d{2}$"))
                .bind(Document::getRegNumber, Document::setRegNumber);
        binder.forField(regDatePicker)
                .withValidator(date -> date != null
&&!date.isAfter(LocalDate.now()),"Дата регистрации не может быть позже
текущей")
                .bind(Document::getRegDate, Document::setRegDate);
        binder.forField(typeField)
                .bind(Document::getType, Document::setType);
        // Обработчики кнопок
        saveButton.addClickListener(event -> saveDocument());
        clearButton.addClickListener(event -> clearForm());
    }
    private void saveDocument() {
        Document document = new Document();
        if (binder.writeBeanIfValid(document)) {
            // Сохраняем через сервис
            documentService.saveDocument(document);
            Notification.show("Документ успешно сохранён: " +
                    document.getTitle());
            clearForm(); // опционально очистить после сохранения
```

```
        } else {
            Notification.show("Пожалуйста, исправьте ошибки в форме", 3000,
                    Notification.Position.MIDDLE);
        }
    }
    private void clearForm() {
        binder.readBean(new Document()); // сбрасываем значения
        // Можно также явно очистить поля, но readBean(new Document())
достаточно
    }
}
```

2.5 Часть 5: Тестирование приложения

- Перезапустите приложение в IntelliJ IDEA. Откройте в браузере адрес: http://localhost:8080/document-form.
- Проверьте работу валидации, попробуйте нажать «Сохранить» с пустыми полями или ввести неверный формат номера (12-34), либо дату из будущего - система должна подсветить ошибки (рисунок 3):

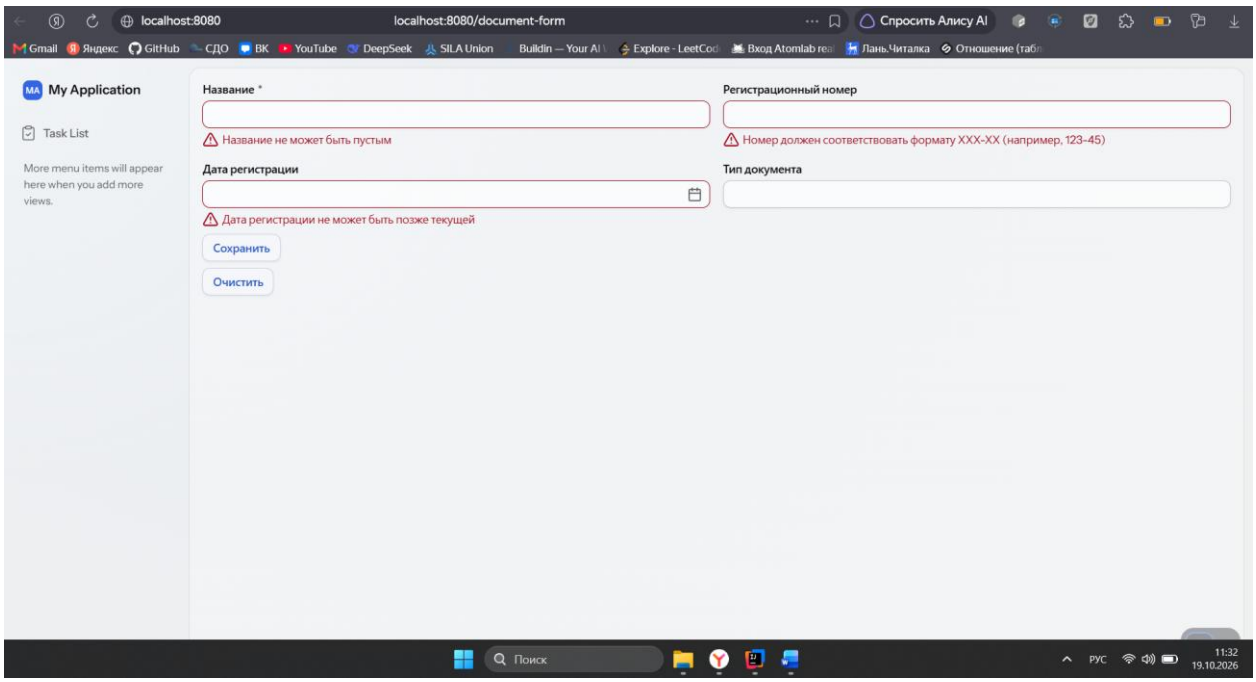


Рисунок 3 – Ошибка валидации

- Введите корректные данные и нажмите «Сохранить»- появится всплывающее уведомление, а объект выведется в консоль IDE (рисунки 4-6):

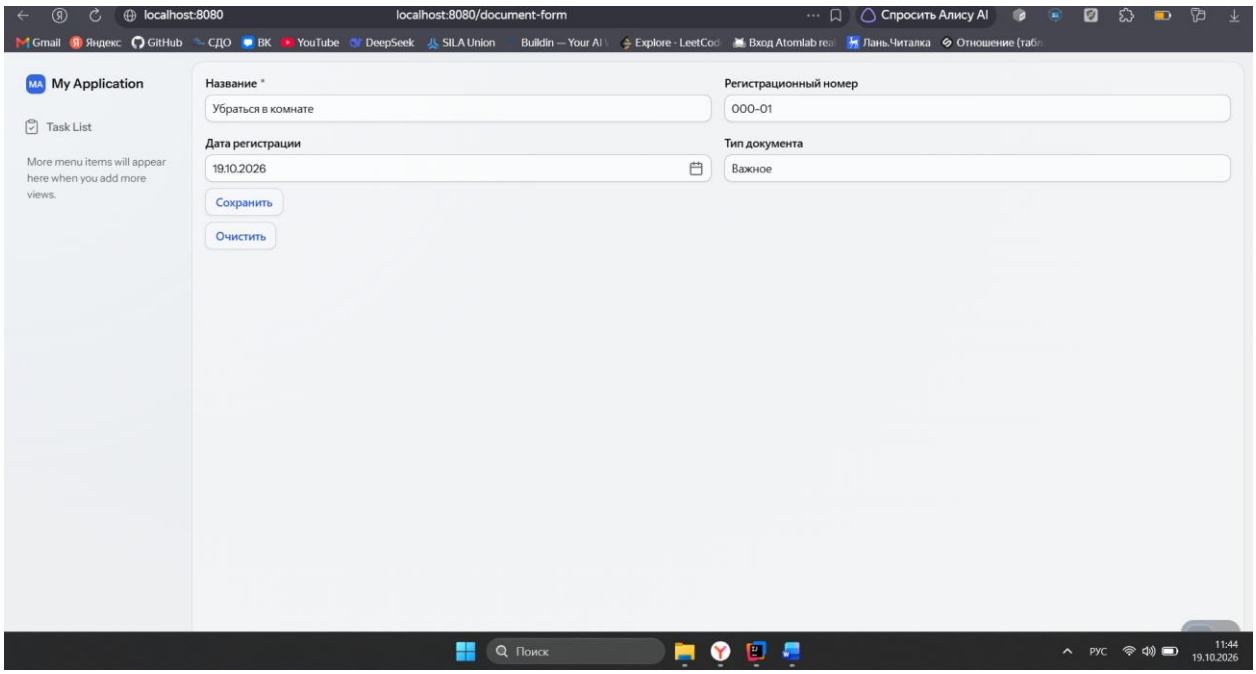


Рисунок 4 – Вводим значения в ячейки

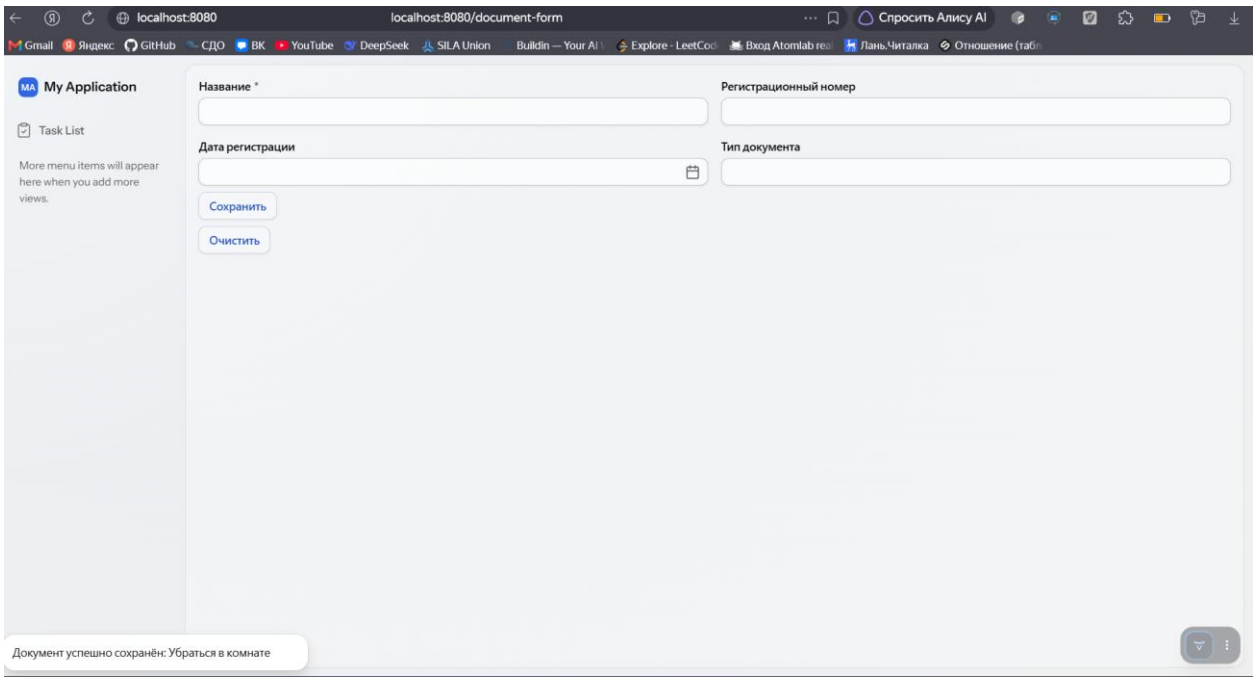


Рисунок 5 – Значение об успешном сохранение документа

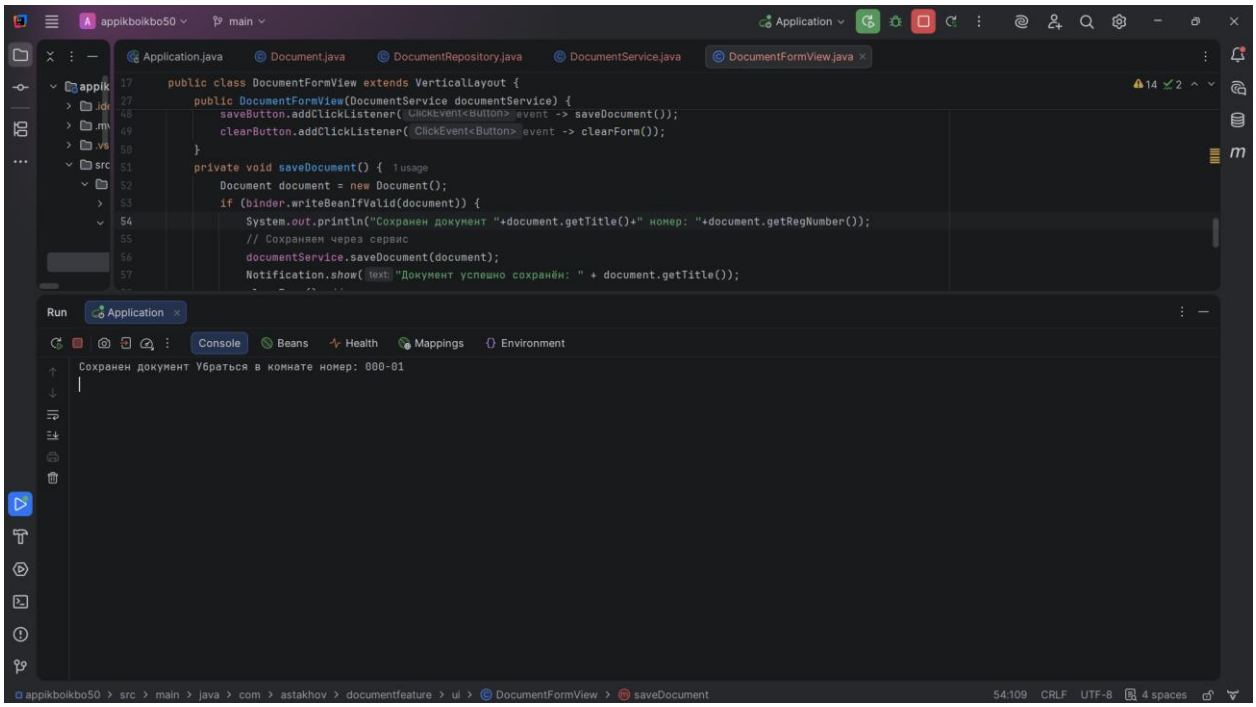


Рисунок 6 – Вывод в консоли intellijIdea

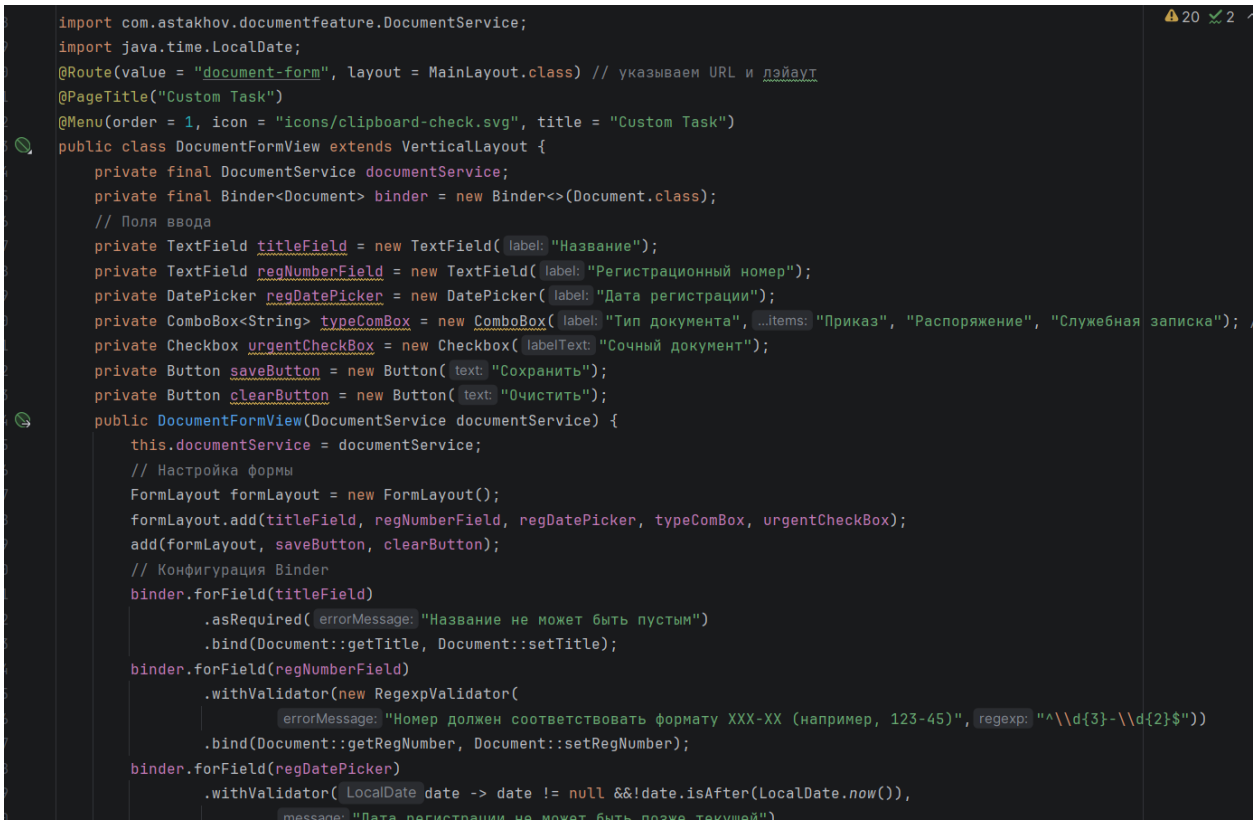
Доработаем созданную форму создания документа:

1. Заменим текстовое поле typeField (TextField) на выпадающий список ComboBox. Зададим ему доступные значения: «Приказ», «Распоряжение», «Служебная записка».

2. Добавим в модель Document новое поле «Срочный документ» (boolean urgent). Добавим на форму соответствующий чекбокс (Checkbox) и настроим его связывание через Binder.

3. Добавим вывод нового поля в логи консоли при успешном сохранении.

Итоговый измененный код, формы с заполненным чекбоксом и выпадающим списком, а также терминал IDEA с информацией о сохраненном документе представлены на рисунках 7-10 :



```
import com.astakhov.documentfeature.DocumentService;
import java.time.LocalDate;
@Route(value = "document-form", layout = MainLayout.class) // указываем URL и лэйаут
@PageTitle("Custom Task")
@Menu(order = 1, icon = "icons/clipboard-check.svg", title = "Custom Task")
public class DocumentFormView extends VerticalLayout {
    private final DocumentService documentService;
    private final Binder<Document> binder = new Binder<>(Document.class);
    // Поля ввода
    private TextField titleField = new TextField( label: "Название");
    private TextField regNumberField = new TextField( label: "Регистрационный номер");
    private DatePicker regDatePicker = new DatePicker( label: "Дата регистрации");
    private ComboBox<String> typeComBox = new ComboBox( label: "Тип документа", ...items: "Приказ", "Распоряжение", "Служебная записка");
    private Checkbox urgentCheckBox = new Checkbox( labelText: "Сочный документ");
    private Button saveButton = new Button( text: "Сохранить");
    private Button clearButton = new Button( text: "Очистить");
    public DocumentFormView(DocumentService documentService) {
        this.documentService = documentService;
        // Настройка формы
        FormLayout formLayout = new FormLayout();
        formLayout.add(titleField, regNumberField, regDatePicker, typeComBox, urgentCheckBox);
        add(formLayout, saveButton, clearButton);
        // Конфигурация Binder
        binder.forField(titleField)
                .asRequired( errorMessage: "Название не может быть пустым")
                .bind(Document::getTitle, Document::setTitle);
        binder.forField(regNumberField)
                .withValidator(new RegexpValidator(
                        errorMessage: "Номер должен соответствовать формату XXX-XX (например, 123-45)", regexp: "^\\d{3}-\\d{2}$"))
                .bind(Document::getRegNumber, Document::setRegNumber);
        binder.forField(regDatePicker)
                .withValidator( LocalDate date -> date != null &&!date.isAfter(LocalDate.now()),
                        message: "Дата регистрации не может быть позже текущей")
```

Рисунок 7 – Добавление checkbox и combobox в класс DocumentFormView.java

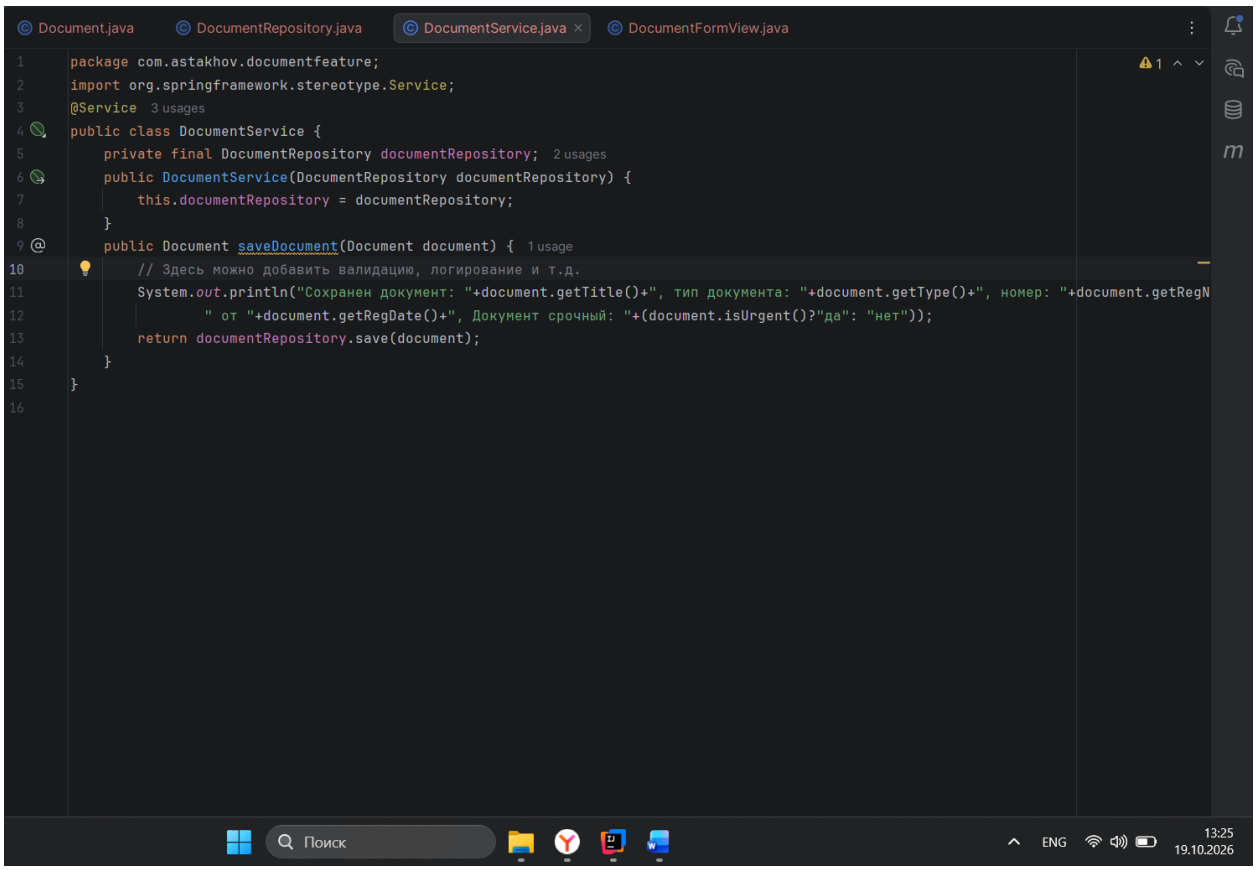


Рисунок 8 – Добавление валидации в консоль

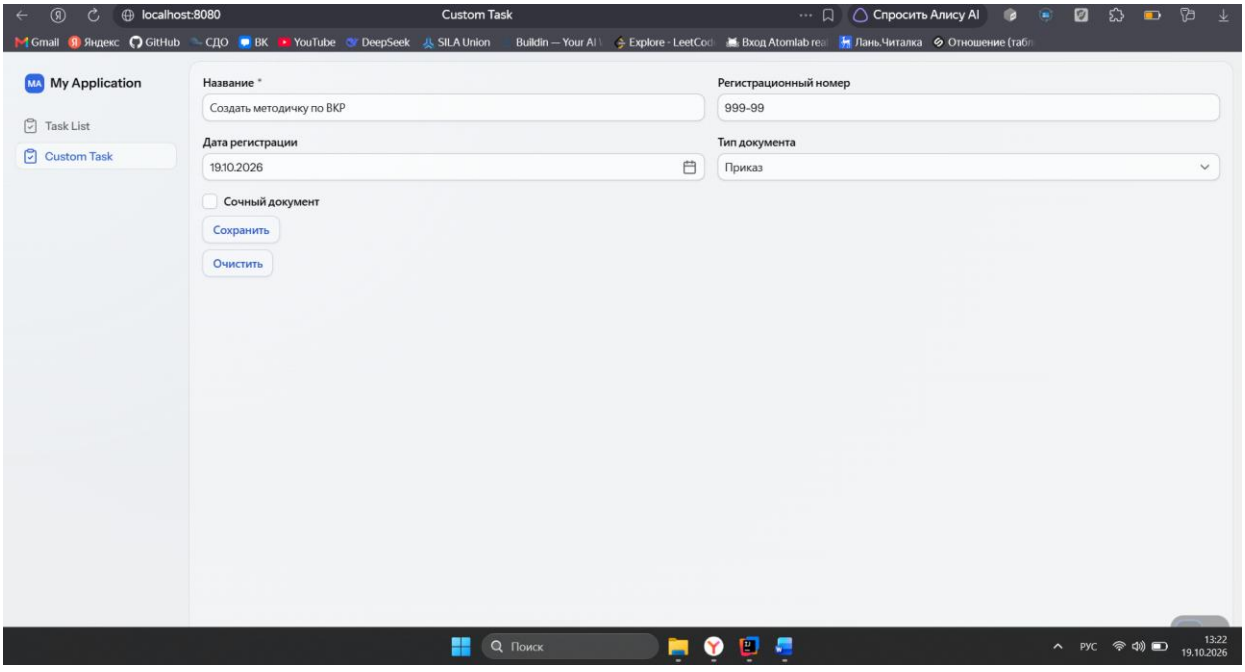


Рисунок 9 – Заполняемые параметры в Custom Task

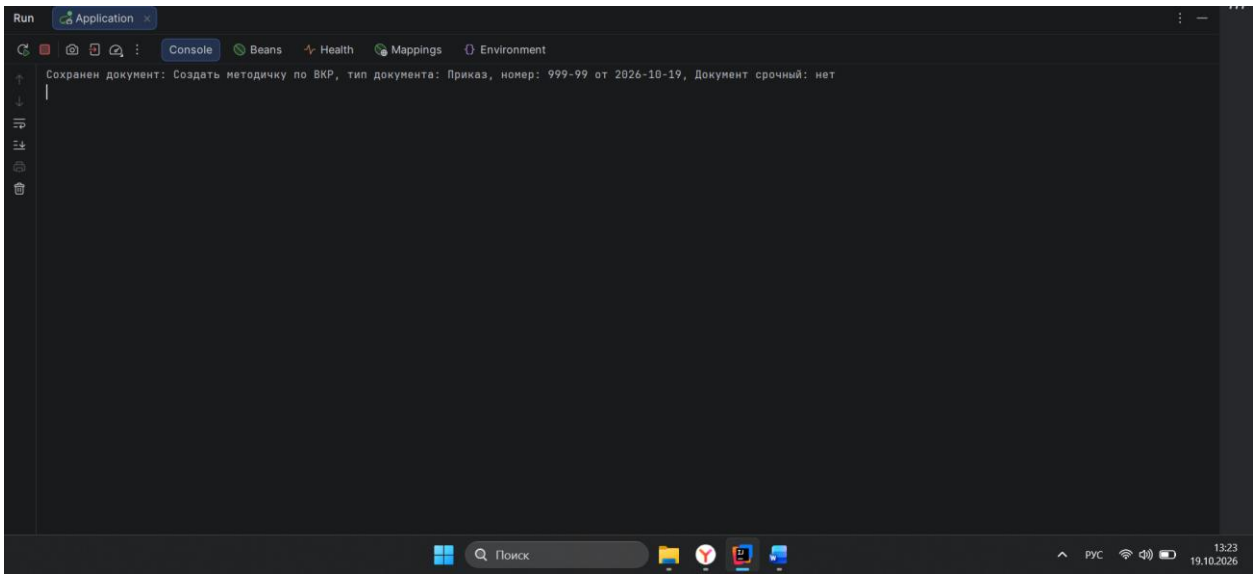


Рисунок 10 – Сообщение в intellijIdea

В ходе самостоятельной работы я добавил панель для переключения созданного нами taskmanager, переделал тип документа, теперь он принимает только три значения, а также добавил checkbox на проверку Срочный документ или нет.

3. Вывод

В ходе выполнения данной практической работы мы ознакомились с фреймворком vaadin. Доработали Taskmanager, а также получили базовые навыки работы с вебсервером.